

Exercise:

Simulate the Best Fit Memory Allocation Algorithm in C following the given steps. Also attach the print out of code and snapshot of output window.

- Declare structures 'hole' and 'process' to hold information about set of holes and processes respectively.
- Get number of holes; say 'nh' and the size of each hole.
- Get number of processes, say 'np'. Also get the memory requirements for each process.
- Allocate processes to holes, by examining each hole as follows:
 - Sort the holes according to their sizes in ascending order.
 - If hole size > process size then mark process as allocated to that hole and decrement hole size by process size.
 - Otherwise check the next from the set of sorted holes.
- Print the list of process and their allocated holes or unallocated status.
- Print the list of holes, their actual and current availability.
- Stop

```
#include <stdio.h>

struct process{
    int size;
    int flag;
    int holeid;
} p[10];

struct hole{
    int size;
    int actual;
    int id;
} h[10];

int main() {
    int i, j, np, nh;
    printf("Enter the number of holes: ");
    scanf("%d", &nh);

    for (i = 0; i < nh; i++) {
        printf("Enter size for hole H%d: ", i);
        scanf("%d", &h[i].size);
        h[i].actual = h[i].size;
        h[i].id = 1;
    }

    printf("Enter number of processes: ");
    scanf("%d", &np);

    for (i = 0; i < np; i++) {
        printf("Enter size of process P%d: ", i);
        scanf("%d", &p[i].size);
        p[i].flag = 0;
    }

    for (i = 0; i < np; i++) {
        int bestIndex = -1;
        for (j = 0; j < nh; j++) {
            if (h[j].size >= p[i].size) {
                if (bestIndex == -1 || h[j].size <
                    h[bestIndex].size) {
                    bestIndex = j;
                }
            }
        }
        if (bestIndex != -1) {
            p[i].flag = 1;
            p[i].holeid = bestIndex;
            h[bestIndex].size -= p[i].size;
        }
        printf("\n BestFit\n");
        printf("\n Process \t PSize \t Hole\n");
        for (i = 0; i < np; i++) {
            if (p[i].flag == 1)
                printf("\n P%d \t %d \t H%d \t %d \t %d",
                    i, p[i].size, h[p[i].holeid].id,
                    p[i].size, h[p[i].holeid].actual);
            printf("\n\n Hole \t Actual \t Available\n");
            for (i = 0; i < nh; i++) {
                printf("\n H%d \t %d \t %d \t %d",
                    i, h[i].id, h[i].actual, h[i].size);
            }
        }
        return 0;
    }
}
```